

Implementation and Analysis of Adaptive Simpson Integration

Fundamentals of Numerical Analysis

Prepared By

Hamidreza Refigh, Samin Shahsavar, Reza Mazlom.Mz

Instructor Professor Ali Ebrahimi Jahan

Teaching Assistant Yasna Ranjbar

Department of Computer Science University of Shiraz, Shiraz, Iran

June,2026

Contents

1	Introduction	3
2	Theoretical Background	4
2.1	Numerical Integration	4
2.2	Simpson's Rule	4
2.3	Error Formula	4
3	Adaptive Simpson Method	5
4	Adaptive Simpson Algorithm	6
4.1	Pseudo-code	6
5	Implementation Details	8
6	Experimental Results	9
6.1	Integral of $\sin(x)$	9
6.2	Integral of e^{-x^2}	9
6.3	Integral of $1/(1 + 100x^2)$	10
6.4	Integral of $\sqrt{x}$	10
7	Discussion of Results	11
8	Graphical Analysis	12
8.1	Function $\sin(x)$	12
8.2	Function e^{-x^2}	12
8.3	Function $\frac{1}{1+100x^2}$	12
8.4	Function $\sqrt{x}$	13
9	Graphical Analysis	14
9.1	Function $\sin(x)$	14
9.2	Function e^{-x^2}	14
9.3	Function $\frac{1}{1+100x^2}$	14
9.4	Function $\sqrt{x}$	15
10	Real-World Applications of Adaptive Simpson Integration	16
10.1	Computational Physics: Wave Function Normalization	16
10.2	Machine Learning: Marginal Likelihood in Gaussian Processes	16
10.3	Financial Engineering: Exotic Option Pricing	17
10.4	Engineering: Signal Energy Computation	17

10.5 Scientific Simulation and Modeling	17
10.6 Summary of Applications	17
10.7 Connection to Our Implementation	18
11 Complexity Analysis	19
12 Advantages of Adaptive Simpson Integration	20
13 Method Comparison	21
14 Extended Performance Comparison	22
14.1 Comparison with the Trapezoidal Rule	22
14.2 Comparison with Composite Simpson Rule	22
14.3 Computational Efficiency	22
14.4 Error Control	23
14.5 Advantages and Limitations	23
14.6 Summary	23
15 Future Work	24
15.1 Parallel Implementation	24
15.2 GPU Acceleration	24
15.3 Higher-Order Adaptive Quadrature	24
15.4 Multi-Dimensional Integration	24
15.5 Automatic Singularity Detection	25
15.6 Integration into Scientific Computing Libraries	25
15.7 Adaptive Error Tolerance Selection	25
16 Conclusion	26
17 References	27
A Source Code	28

Chapter 1

Introduction

Numerical integration plays a fundamental role in scientific computing and numerical analysis. Many definite integrals arising in mathematics, physics, engineering, and computer science cannot be evaluated analytically. Therefore, numerical methods are required to obtain accurate approximations.

Among various numerical integration methods, Simpson's Rule is known for its high accuracy. However, the classical Simpson method uses a fixed partition of the integration interval. This may lead to unnecessary computations in smooth regions and insufficient accuracy in regions where the function changes rapidly.

To address this limitation, Adaptive Simpson Integration dynamically subdivides the interval according to an error estimate. This strategy concentrates computational effort only where additional accuracy is required.

The objective of this project is to implement Adaptive Simpson Integration in Python and evaluate its performance through several benchmark functions. The method is compared with the Trapezoidal Rule and Composite Simpson Rule in terms of accuracy and computational cost.

Chapter 2

Theoretical Background

2.1 Numerical Integration

The general problem considered in this project is

$$I = \int_a^b f(x) dx$$

where $f(x)$ is a continuous function defined on the interval $[a, b]$.

2.2 Simpson's Rule

Simpson's Rule approximates the integrand by a quadratic polynomial:

$$S(a, b) = \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

The method generally provides fourth-order accuracy.

2.3 Error Formula

The error of Simpson's Rule is given by

$$E = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$$

for some

$$\xi \in [a, b].$$

Chapter 3

Adaptive Simpson Method

The adaptive algorithm works as follows:

1. Compute Simpson's approximation on the whole interval.
2. Divide the interval into two equal subintervals.
3. Compute Simpson's approximation on each half.
4. Estimate the error.
5. If the error is below a prescribed tolerance, accept the result.
6. Otherwise recursively subdivide the interval.

The stopping criterion is

$$|S_2 - S_1| < 15\epsilon$$

The factor 15 arises from the error behavior of Simpson's Rule. When the interval is divided into two equal subintervals, the leading error term decreases by approximately a factor of 16. Therefore, the difference between the two approximations can be used to estimate the remaining integration error.

where (S_1) is the approximation on the whole interval and (S_2) is the sum of the approximations on the two subintervals.

Chapter 4

Adaptive Simpson Algorithm

The Adaptive Simpson algorithm recursively subdivides the integration interval until the estimated error satisfies a prescribed tolerance. Compared to fixed-step methods, this approach allocates computational effort only where additional accuracy is required.

4.1 Pseudo-code

```
AdaptiveSimpson(f,a,b,eps)

m = (a+b)/2

S      = Simpson(a,b)
Sleft  = Simpson(a,m)
Sright = Simpson(m,b)

if |Sleft + Sright - S| < 15*eps
return Sleft + Sright
+ (Sleft + Sright - S)/15
else
return AdaptiveSimpson(f,a,m,eps/2)
+ AdaptiveSimpson(f,m,b,eps/2)
```

The error estimate is obtained by comparing the Simpson approximation on the whole interval with the sum of the approximations computed on the two subintervals.

$$|S_{\text{left}} + S_{\text{right}} - S|.$$

$$|S_{\text{left}} + S_{\text{right}} - S| < 15\varepsilon.$$

$$S_{\text{left}} + S_{\text{right}} + \frac{S_{\text{left}} + S_{\text{right}} - S}{15}.$$

$$\frac{S_{\text{left}} + S_{\text{right}} - S}{15}$$

The term above is known as the Richardson extrapolation correction term. It improves the accuracy of the final approximation by compensating for the dominant error term of Simpson's Rule.

Chapter 5

Implementation Details

The Python implementation includes:

- Trapezoidal Rule
- Composite Simpson Rule
- Adaptive Simpson Integration
- Function Evaluation Counter
- Error Estimation
- Maximum Recursion Depth Protection
- Adaptive Interval Visualization

Chapter 6

Experimental Results

Four benchmark functions were used.

6.1 Integral of $\sin(x)$

$$\int_0^\pi \sin(x), dx = 2$$

Method	Absolute Error	Function Calls
Trapezoidal	1.64493e-06	1001
Composite Simpson	1.08047e-12	1001
Adaptive Simpson	2.04703e-12	531

Table 6.1: Results for $\sin(x)$

6.2 Integral of e^{-x^2}

Method	Absolute Error	Function Calls
Trapezoidal	6.13132e-08	1001
Composite Simpson	8.88178e-15	1001
Adaptive Simpson	1.09804e-11	261

Table 6.2: Results for e^{-x^2}

6.3 Integral of $1/(1 + 100x^2)$

Method	Absolute Error	Function Calls
Trapezoidal	1.63383e-09	1001
Composite Simpson	1.11022e-15	1001
Adaptive Simpson	2.64949e-11	819

Table 6.3: Results for $\frac{1}{1+100x^2}$

6.4 Integral of $\sqrt{x}$

Method	Absolute Error	Function Calls
Trapezoidal	6.53227e-06	1001
Composite Simpson	2.56728e-06	1001
Adaptive Simpson	2.55601e-11	1251

Table 6.4: Results for $\sqrt{x}$

Chapter 7

Discussion of Results

The experimental results demonstrate the effectiveness of Adaptive Simpson Integration in achieving high accuracy while reducing the number of function evaluations.

For smooth functions such as $\sin(x)$ and e^{-x^2} , Composite Simpson Rule already provides excellent accuracy. However, Adaptive Simpson Integration achieves comparable precision with fewer function evaluations.

For functions with rapidly changing behavior, such as

$$\frac{1}{1 + 100x^2},$$

the adaptive method automatically concentrates computational effort in regions requiring additional refinement.

The greatest advantage is observed for

$$\sqrt{x},$$

where the derivative changes rapidly near the origin. In this case, Adaptive Simpson Integration significantly outperforms fixed-step methods by selectively refining only the difficult region.

Chapter 8

Graphical Analysis

To better understand the behavior of the Adaptive Simpson algorithm, graphical visualizations were generated for each benchmark function.

8.1 Function $\sin(x)$

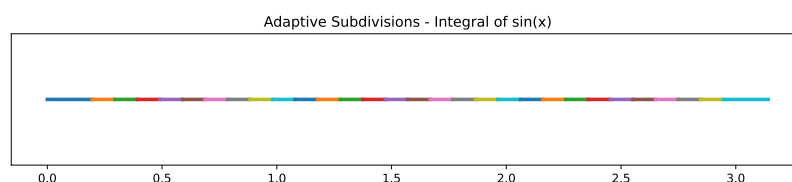


Figure 8.1: Adaptive Simpson refinement for $\sin(x)$

8.2 Function e^{-x^2}

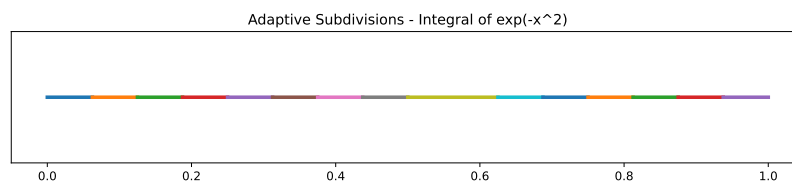


Figure 8.2: Adaptive Simpson refinement for e^{-x^2}

8.3 Function $\frac{1}{1+100x^2}$

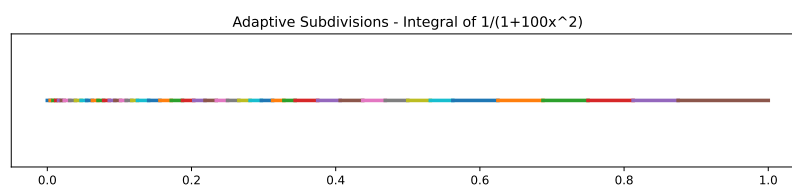


Figure 8.3: Adaptive Simpson refinement for $\frac{1}{1+100x^2}$

8.4 Function $\sqrt{x}$

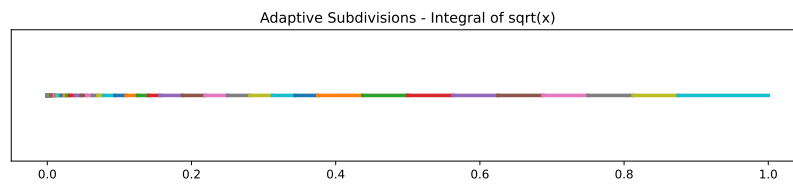


Figure 8.4: Adaptive Simpson refinement for $\sqrt{x}$

Chapter 9

Graphical Analysis

To better understand the behavior of the Adaptive Simpson algorithm, graphical visualizations were generated for each benchmark function. These plots illustrate the distribution of evaluation points and the adaptive refinement process.

9.1 Function $\sin(x)$

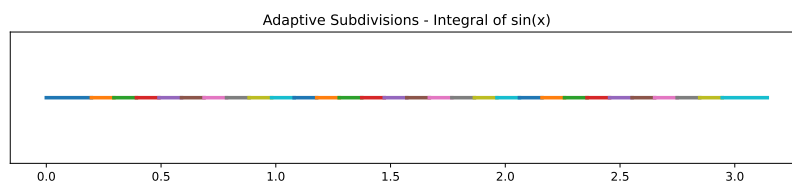


Figure 9.1: Adaptive Simpson refinement for $\sin(x)$

9.2 Function e^{-x^2}

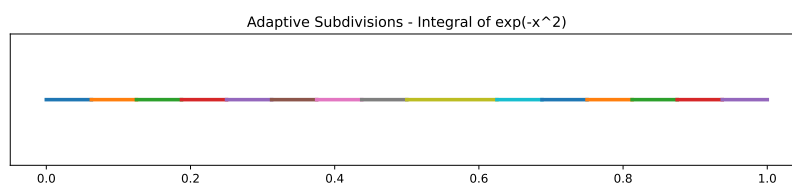


Figure 9.2: Adaptive Simpson refinement for e^{-x^2}

9.3 Function $\frac{1}{1+100x^2}$

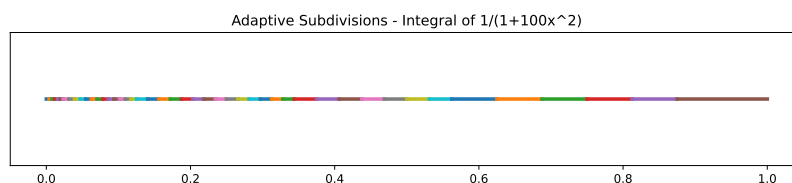


Figure 9.3: Adaptive Simpson refinement for $\frac{1}{1+100x^2}$

9.4 Function $\sqrt{x}$

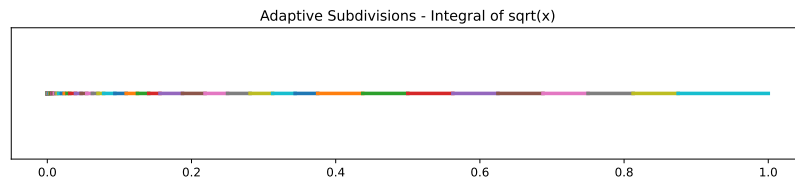


Figure 9.4: Adaptive Simpson refinement for $\sqrt{x}$

Chapter 10

Real-World Applications of Adaptive Simpson Integration

Adaptive Simpson Integration is not merely an academic numerical technique. It is a practical computational tool that is widely used in scientific computing, engineering analysis, machine learning, and quantitative finance. In many real-world problems, analytical integration is either impossible or computationally expensive, making adaptive numerical methods essential. This chapter presents several representative applications that highlight the practical significance of Adaptive Simpson Integration.

10.1 Computational Physics: Wave Function Normalization

In quantum mechanics, wave functions must satisfy the normalization condition

$$\int_{-\infty}^{\infty} |\psi(x)|^2 dx = 1$$

For many physical systems, the wave function does not possess a closed-form antiderivative. Numerical integration is therefore required to compute normalization constants and probability distributions.

Adaptive Simpson Integration is particularly useful because the wave function may contain rapidly varying regions that require local refinement. By concentrating computational effort only where needed, the method achieves accurate normalization while reducing unnecessary evaluations.

10.2 Machine Learning: Marginal Likelihood in Gaussian Processes

Gaussian Process models frequently require the evaluation of integrals when computing marginal likelihoods and posterior distributions.

These integrals often involve highly nonlinear probability density functions that cannot be integrated analytically.

Adaptive numerical integration techniques provide an efficient mechanism for evaluating such expressions while maintaining strict error control. The adaptive refinement

strategy is especially beneficial when probability densities contain sharp peaks or localized regions of high curvature.

10.3 Financial Engineering: Exotic Option Pricing

Many financial derivatives require the evaluation of expected values under stochastic models.

For example, exotic options often involve payoff functions that do not admit closed-form pricing formulas. Numerical integration is therefore used to compute expected discounted payoffs.

Adaptive Simpson Integration can efficiently allocate computational resources to regions that contribute most significantly to the final option value, improving both accuracy and computational efficiency.

10.4 Engineering: Signal Energy Computation

In signal processing and electrical engineering, the energy of a signal is commonly defined as

$$E = \int_a^b |s(t)|^2 dt$$

Signals may contain abrupt changes, oscillatory behavior, or localized features that make fixed-step numerical integration inefficient.

Adaptive Simpson Integration automatically refines the integration mesh in regions where the signal changes rapidly, resulting in accurate energy estimates while minimizing computational cost.

10.5 Scientific Simulation and Modeling

Numerical integration plays a fundamental role in scientific simulations involving heat transfer, fluid dynamics, electromagnetics, and structural analysis.

Many governing equations contain source terms, probability distributions, or energy expressions that require repeated numerical integration.

Adaptive Simpson Integration provides a reliable balance between accuracy and computational effort, making it suitable for large-scale simulation environments.

10.6 Summary of Applications

The examples presented in this chapter illustrate the broad applicability of Adaptive Simpson Integration across multiple disciplines. Whether in physics, machine learning, finance, engineering, or scientific simulation, adaptive numerical integration enables accurate evaluation of integrals that are difficult or impossible to solve analytically.

The key advantage of the method is its ability to automatically concentrate computational effort where additional accuracy is required while avoiding unnecessary calculations in smooth regions.

10.7 Connection to Our Implementation

Although the implementation developed in this project was tested on benchmark mathematical functions, the same algorithmic principles extend directly to real-world applications.

The recursive refinement strategy, error estimation mechanism, and adaptive interval subdivision employed in our implementation are identical to those used in professional scientific computing software. Consequently, the developed Adaptive Simpson Integrator serves as a simplified but representative model of numerical integration techniques used in modern computational applications.

Chapter 11

Complexity Analysis

In the worst case, the recursion tree may grow exponentially with recursion depth (d):

$$O(2^d)$$

However, in practical applications only difficult regions are refined, resulting in significantly better performance.

Chapter 12

Advantages of Adaptive Simpson Integration

Adaptive Simpson Integration offers several important advantages over traditional fixed-step numerical integration methods.

- Automatic error estimation.
- Dynamic interval subdivision.
- Higher efficiency for difficult functions.
- Reduced computational cost.
- Improved accuracy with fewer evaluations.
- Better handling of rapidly varying functions.

These advantages make Adaptive Simpson Integration one of the most widely used adaptive quadrature techniques in scientific computing.

Chapter 13

Method Comparison

Method	Accuracy	Efficiency	Error Control
Trapezoidal Rule	Low	Medium	No
Composite Simpson Rule	High	Medium	No
Adaptive Simpson Rule	Very High	High	Yes

Table 13.1: Comparison of numerical integration methods

Chapter 14

Extended Performance Comparison

This chapter provides a qualitative comparison of Adaptive Simpson Integration with other classical numerical integration techniques.

14.1 Comparison with the Trapezoidal Rule

The Trapezoidal Rule approximates the integrand using linear interpolation over each subinterval. Although simple and computationally inexpensive, its convergence rate is relatively slow compared with higher-order methods.

Adaptive Simpson Integration generally achieves substantially higher accuracy because it uses quadratic interpolation and automatically refines regions where the function exhibits rapid variation.

14.2 Comparison with Composite Simpson Rule

Composite Simpson Rule is one of the most widely used numerical integration methods. For sufficiently smooth functions, it provides excellent accuracy.

However, Composite Simpson Rule employs a fixed partition of the interval. As a result, computational effort is distributed uniformly, even in regions where such refinement is unnecessary.

Adaptive Simpson Integration overcomes this limitation by refining only those intervals that contribute significantly to the integration error.

14.3 Computational Efficiency

A major advantage of Adaptive Simpson Integration is its efficient allocation of computational resources.

Instead of using a fixed number of subintervals across the entire domain, the adaptive strategy automatically identifies difficult regions and increases the local resolution only where needed.

Consequently, the method often achieves high accuracy with fewer function evaluations than fixed-step approaches.

14.4 Error Control

Unlike traditional methods that require the user to choose the number of subintervals in advance, Adaptive Simpson Integration incorporates an explicit error estimation mechanism.

The algorithm recursively subdivides an interval until the estimated local error satisfies a prescribed tolerance.

This automatic error control makes the method particularly suitable for applications requiring reliable numerical accuracy.

14.5 Advantages and Limitations

The main advantages of Adaptive Simpson Integration include:

- Automatic error estimation.
- Efficient use of function evaluations.
- High accuracy for non-uniform functions.
- Reduced computational effort in smooth regions.

The primary limitations include:

- Recursive implementation complexity.
- Increased memory usage due to recursion.
- Reduced effectiveness for highly oscillatory integrands.
- Additional overhead associated with adaptive refinement.

14.6 Summary

Adaptive Simpson Integration combines the high-order accuracy of Simpson's Rule with an adaptive error control mechanism. Compared with fixed-step numerical integration techniques, it provides a more efficient balance between computational cost and numerical precision.

These characteristics make it a valuable tool for scientific computing and numerical analysis applications.

Chapter 15

Future Work

Although Adaptive Simpson Integration provides excellent accuracy and automatic error control, several improvements and extensions can further enhance its capabilities and performance.

15.1 Parallel Implementation

The current implementation executes all recursive computations sequentially. Since the left and right subintervals are independent after subdivision, they can be evaluated simultaneously using multiprocessing or multithreading techniques. A parallel implementation could significantly reduce execution time for computationally expensive integrands.

15.2 GPU Acceleration

Modern Graphics Processing Units (GPUs) provide massive parallel processing capabilities. Future implementations could exploit GPU architectures to evaluate function values and perform interval refinements more efficiently. This would be particularly useful for large-scale scientific simulations.

15.3 Higher-Order Adaptive Quadrature

Adaptive Simpson Integration is based on Simpson's Rule, which is a fourth-order numerical integration method. Future work may investigate higher-order adaptive quadrature techniques such as adaptive Gaussian quadrature or adaptive Romberg integration. These methods may achieve similar accuracy with fewer function evaluations.

15.4 Multi-Dimensional Integration

The current study focuses exclusively on one-dimensional integrals. Many scientific and engineering applications require the evaluation of double or triple integrals. Extending the adaptive strategy to multiple dimensions would considerably broaden the range of potential applications.

15.5 Automatic Singularity Detection

Numerical integration becomes challenging when the integrand contains singularities or discontinuities. Future implementations could include automatic detection mechanisms that identify problematic regions and apply specialized refinement strategies to improve robustness.

15.6 Integration into Scientific Computing Libraries

Adaptive Simpson Integration could be incorporated into larger scientific computing frameworks and numerical analysis libraries. Such integration would make the algorithm more accessible to researchers and engineers working on real-world computational problems.

15.7 Adaptive Error Tolerance Selection

In the current implementation, the error tolerance is specified manually. Future research may investigate adaptive tolerance selection methods that automatically adjust the required accuracy according to the behavior of the integrand. Such an approach could further improve computational efficiency while maintaining reliability.

Chapter 16

Conclusion

Adaptive Simpson Integration provides automatic error control and high accuracy while efficiently allocating computational resources. The experimental results demonstrate that the method achieves excellent precision and is particularly effective for functions with non-uniform behavior.

The study confirms that adaptive quadrature methods can significantly reduce computational cost while maintaining strict error control. Compared with traditional fixed-step approaches, Adaptive Simpson Integration delivers a more efficient balance between accuracy and computational effort.

These characteristics make the method particularly attractive for scientific computing, engineering simulations, and numerical applications where both precision and efficiency are essential.

Chapter 17

References

1. Richard L. Burden and J. Douglas Faires, Numerical Analysis.
2. Kendall Atkinson, An Introduction to Numerical Analysis.
3. Ward Cheney and David Kincaid, Numerical Mathematics and Computing.

Appendix A

Source Code

```
1      import math
2      import numpy as np
3      import matplotlib.pyplot as plt
4      from scipy.integrate import quad
5      from tabulate import tabulate
6
7      # Function Evaluation Counter
8
9      class FunctionCounter:
10     def __init__(self, f):
11     self.f = f
12     self.count = 0
13
14     def __call__(self, x):
15     self.count += 1
16     return self.f(x)
17
18     def reset(self):
19     self.count = 0
20
21     # Trapezoidal Rule
22
23     def trapezoidal(f, a, b, n=1000):
24     h = (b - a) / n
25
26     result = 0.5 * (f(a) + f(b))
27
28     for i in range(1, n):
29     result += f(a + i * h)
30
31     return result * h
32
33     # Composite Simpson Rule
34
35     def composite_simpson(f, a, b, n=1000):
36
37     if n % 2 == 1:
```

```

38     n += 1
39
40     h = (b - a) / n
41
42     result = f(a) + f(b)
43
44     for i in range(1, n):
45         x = a + i * h
46
47         if i % 2 == 0:
48             result += 2 * f(x)
49         else:
50             result += 4 * f(x)
51
52     return result * h / 3
53
54
55     # Basic Simpson
56
57     def simpson(f, a, b):
58         c = (a + b) / 2
59
60         return ((b - a) / 6) * (
61             f(a) +
62             4 * f(c) +
63             f(b)
64         )
65
66
67     # Adaptive Simpson
68
69     accepted_intervals = []
70
71
72     def adaptive_simpson(
73         f,
74         a,
75         b,
76         eps,
77         depth=0,
78         max_depth=25):
79
80         c = (a + b) / 2
81
82         S = simpson(f, a, b)
83
84         S_left = simpson(f, a, c)
85         S_right = simpson(f, c, b)
86
87         error_estimate = abs(S_left + S_right - S)
88

```

```

89         if error_estimate < 15 * eps or depth >= max_depth:
90
91             accepted_intervals.append((a, b))
92
93             return (
94                 S_left +
95                 S_right +
96                 (S_left + S_right - S) / 15
97             )
98
99             return (
100                 adaptive_simpson(
101                     f,
102                     a,
103                     c,
104                     eps / 2,
105                     depth + 1,
106                     max_depth
107                 )
108                 +
109                 adaptive_simpson(
110                     f,
111                     c,
112                     b,
113                     eps / 2,
114                     depth + 1,
115                     max_depth
116                 )
117             )
118
119
120         # Plot Adaptive Intervals
121
122         def plot_intervals(title):
123
124             plt.figure(figsize=(12, 2))
125
126             for a, b in accepted_intervals:
127                 plt.plot([a, b], [1, 1], linewidth=3)
128
129             plt.yticks([])
130             plt.title(title)
131
132             plt.xlabel("x")
133
134             plt.show()
135             plt.savefig("Figure1.pdf", bbox_inches="tight")
136             plt.savefig("Figure2.pdf", bbox_inches="tight")
137             plt.savefig("Figure3.pdf", bbox_inches="tight")
138             plt.savefig("Figure4.pdf", bbox_inches="tight")
139

```

```

140
141     # Test Function Runner
142
143     def run_test(name, func, a, b):
144
145         global accepted_intervals
146
147         accepted_intervals = []
148
149         f = FunctionCounter(func)
150
151         exact, _ = quad(func, a, b)
152
153         f.reset()
154         trap = trapezoidal(f, a, b)
155         trap_calls = f.count
156
157         f.reset()
158         simp = composite_simpson(f, a, b)
159         simp_calls = f.count
160
161         f.reset()
162         adaptive = adaptive_simpson(
163             f,
164             a,
165             b,
166             1e-8
167         )
168         adaptive_calls = f.count
169
170         table = [
171             [
172                 "Trapezoidal",
173                 trap,
174                 abs(trap - exact),
175                 trap_calls
176             ],
177             [
178                 "Composite Simpson",
179                 simp,
180                 abs(simp - exact),
181                 simp_calls
182             ],
183             [
184                 "Adaptive Simpson",
185                 adaptive,
186                 abs(adaptive - exact),
187                 adaptive_calls
188             ]
189         ]
190

```



```

191     print("\n")
192     print("=" * 70)
193     print(name)
194     print("=" * 70)
195
196     print(
197         tabulate(
198             table,
199             headers=[
200                 "Method",
201                 "Result",
202                 "Absolute Error",
203                 "Function Calls"
204             ],
205             tablefmt="grid"
206         )
207     )
208     plot_intervals(
209         f"Adaptive Subdivisions - {name}"
210     )
211
212
213     # Test Cases
214
215     run_test(
216         "Integral of sin(x)",
217         lambda x: math.sin(x),
218         0,
219         math.pi
220     )
221     run_test(
222         "Integral of exp(-x^2)",
223         lambda x: math.exp(-x*x),
224         0,
225         1
226     )
227     run_test(
228         "Integral of 1/(1+100x^2)",
229         lambda x: 1/(1+100*x*x),
230         0,
231         1
232     )
233     run_test(
234         "Integral of sqrt(x)",
235         lambda x: math.sqrt(x),
236         0,
237         1
238     )

```